

Bonus Task Report

LLM Selection

LLM1: GPT 5.2

LLM2: Claude Sonnet 4.5

File Selection

A single Java source file, `ObjectPermission.java`, from the Apache Roller codebase was selected. Using a single file ensured a controlled comparison and avoided speculative cross-file refactorings.

Prompt given to LLMs in Non-agentic mode:

I have a Java file from the Apache Roller blogging platform project.

TASK 1 - DESIGN SMELL IDENTIFICATION:

Analyze this code and identify all design smells present.

For each smell you identify:

1. State the smell name and the specific category it belongs to (abstraction, modularization, encapsulation, hierarchy)
2. Explain specifically where it occurs (class/method name)

TASK 2 - REFACTORING PROPOSAL:

For each identified design smell, propose a specific refactoring strategy:

- Name the refactoring pattern
- Explain how it addresses the smell

TASK 3 - CODE REFACTORING:

Provide refactored code that:

- Addresses ALL identified design smells
- Preserves the original behavior exactly

Here is the code to analyze:

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. The ASF licenses this file to You
 * under the Apache License, Version 2.0 (the "License"); you may not
 * use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
```

```
* distributed under the License is distributed on an "AS IS" BASIS,  
* WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.  
* See the License for the specific language governing permissions and  
* limitations under the License. For additional information regarding  
* copyright in this work, please see the NOTICE file in the top level  
* directory of this distribution.  
*/
```

```
package org.apache.roller.weblogger.pojos;
```

```
import java.util.Date;  
import org.apache.roller.util.UUIDGenerator;
```

```
/**  
 * Base permission class for Roller.  
 */  
public abstract class ObjectPermission extends RollerPermission {  
    protected String id = UUIDGenerator.generateUUID();  
    protected String userName;  
    protected String objectType;  
    protected String objectId;  
    protected boolean pending = false;  
    protected Date dateCreated = new Date();  
    protected String actions;  
  
    public ObjectPermission() {  
        super("");  
    }  
  
    public ObjectPermission(String name) {  
        super(name);  
    }  
    public String getId() {  
        return id;  
    }  
  
    public void setId(String id) {  
        this.id = id;  
    }  
  
    @Override  
    public void setActions(String actions) {  
        this.actions = actions;  
    }  
  
    @Override
```

```

public String getActions() {
    return actions;
}

public String getUsername() {
    return userName;
}

public void setUsername(String username) {
    this.userName = username;
}

/*public String getObjectType() {
    return objectType;
}

public void setObjectType(String objectType) {
    this.objectType = objectType;
}*/

public String getObjectId() {
    return objectId;
}

public void setObjectId(String objectId) {
    this.objectId = objectId;
}

public Date getDateCreated() {
    return dateCreated;
}

public void setDateCreated(Date dateCreated) {
    this.dateCreated = dateCreated;
}

public boolean isPending() {
    return pending;
}

public void setPending(boolean pending) {
    this.pending = pending;
}

}

```

Please provide:

1. A detailed list of design smells found

2. Your refactoring strategy for each
3. The complete refactored code
4. A brief summary of improvements made

Prompts given to LLMs in Agentic mode:

Prompt1:

I have a Java file from the Apache Roller blogging platform project.

File path: roller\app\src\main\java\org\apache\roller\weblogger\pojos\ObjectPermission.java

Follow the steps below sequentially.

TASK 1 – DESIGN SMELL IDENTIFICATION

Analyze the file and identify all design-level smells present.

For each smell:

- State the smell name
- Specify its category (Abstraction / Modularization / Encapsulation / Hierarchy)
- Explain exactly where it occurs (class or method)
- Rate severity (High / Medium / Low)

Prompt2:

TASK 2 – REFACTORING PLAN

For each identified smell:

- Name the refactoring pattern to apply
- Explain how it resolves the smell
- Specify if new classes or interfaces are required

Prompt3:

TASK 3 – IMPLEMENTATION

Generate complete refactored code that:

- Addresses ALL identified design smells
- Preserves original behavior exactly
- Maintains compatibility with the Apache Roller codebase
- If new classes/interfaces are required, provide them.

Link to the responses given by LLMs:

https://docs.google.com/document/d/1zzGQZxezjY91_xMiTxWc4tpv36hNZVsJEGNEQm7ZUT0/edit?usp=sharing

Manual Analysis

From our manual analysis (Task 2A), the permission subsystem exhibited:

- **Broken Hierarchy** (WeblogPermission / ObjectPermission / RollerPermission)
- **Deficient Encapsulation**
- **Cyclic-Dependent Modularization**
- Inconsistent encapsulation policy
- Inheritance contract violations

At the file level (ObjectPermission.java), the manually identified smells included:

1. Deficient Encapsulation (protected state exposure)
2. Incomplete abstraction (commented objectType accessors)
3. Inheritance fragility within permission hierarchy

GPT-5.2 Non-Agent Mode

Design Smell Identification

GPT-5.2 in non-agent mode successfully identified four design smells, demonstrating strong file-level pattern recognition.

- The LLM correctly detected Protected Data, which it categorized under Encapsulation, recognizing that the seven protected fields expose internal state to subclasses and enable direct manipulation without validation. This aligns precisely with our manual identification of Deficient Encapsulation.
- GPT-5.2 also identified Exposed Mutable State, specifically noting that the getDateCreated and setDateCreated methods expose a mutable Date object, allowing external callers to modify internal state without calling setters, creating temporal coupling issues.
- Additionally, GPT-5.2 detected Commented-Out Code, categorized under Modularization, correctly identifying the commented getObjectType and setObjectType methods as incomplete API design that creates maintenance liability and obscures the module's real API.
- The model also flagged Scattered Initialization as an Abstraction smell, noting that field initialization logic is distributed between inline declarations and constructors, making object construction rules less clear and harder to control consistently.

However, GPT-5.2 in this mode did not identify the broader Broken Hierarchy issue within the permission inheritance chain, nor did it detect the Cyclic-Dependent Modularization problem present at the subsystem level.

This indicates that while GPT-5.2 performed exceptionally well at identifying local, file-scoped design issues, it lacked the architectural awareness necessary to detect higher-level structural problems that span multiple files and subsystems. The model's analysis was confined to what could be inferred from a single file in isolation.

Refactoring Quality and Compilability

GPT-5.2 produced syntactically valid and compilable code that directly addressed all four identified smells.

- The refactoring converted all seven protected fields to private, preventing direct subclass access and enforcing encapsulation.
- The model introduced defensive copying for the Date field, implementing a pattern where getDateCreated returns a copy of the internal Date object and

`setDateCreated` accepts a reference but stores a copy, preventing external mutation from changing internal state.

- GPT-5.2 also restored the commented-out `objectType` accessors as functioning methods, completing the API.
- It centralized initialization logic by moving default values into a single helper method invoked by constructors.

The refactored code preserved the public API completely, maintaining all existing method signatures and constructor definitions without introducing cross-file dependencies.

However, there is a critical architectural safety concern:

- Changing protected fields to private may break existing subclasses such as `WeblogPermission` that directly access these fields rather than using getters and setters.

This is a common trade-off in encapsulation refactoring: while the change improves design quality, it risks violating existing inheritance assumptions. The code is technically clean and compilable, but its safety depends entirely on whether subclasses have been properly designed to use accessor methods rather than direct field access.

GPT-5.2 Agentic Mode

Design Smell Identification

Operating in agentic mode, GPT-5.2 demonstrated significantly enhanced analytical depth through multi-step reasoning and iterative inspection. The model identified seven design smells, substantially more than in non-agentic mode, by leveraging its ability to examine related files and perform cross-class analysis.

Beyond the four smells detected in non-agentic mode, the agentic workflow enabled GPT-5.2 to identify:

- Commented-Out Code, specifically for the dead `objectType` interface.
- Potentially Invalid or Meaningless Default Superclass State, referring to the empty string passed to the `RollerPermission` superclass constructor.
- Hard-Coded Dependency, from the construction side effects of field initializers generating UUIDs and timestamps at declaration time.

Most significantly, the agentic mode enabled GPT-5.2 to examine the parent class `RollerPermission` and subclass `WeblogPermission`, allowing it to detect issues at the hierarchy level that were invisible in single-file analysis.

- The model identified that `ObjectPermission` passes an empty string to `RollerPermission`'s constructor, creating permissions with meaningless default names.

- It recognized that field initializers such as UUID generation and Date instantiation create tight coupling to static utilities and make object construction non-deterministic and time-dependent, which complicates testing and persistence rehydration.

These findings demonstrate that agentic workflows substantially improve the architectural awareness of LLMs, enabling them to detect design problems that require understanding relationships between multiple classes.

Refactoring Quality and Compilability

The agentic GPT-5.2 workflow produced comprehensive refactorings that addressed all seven identified smells while ensuring build compatibility through iterative testing.

The model updated `ObjectPermission` by:

- Encapsulating state with private fields and controlled access methods.
- Removing commented dead code.
- Implementing defensive copying for `Date` fields.
- Moving UUID generation and default value initialization to constructor bodies to eliminate side effects.

Critically, the agentic workflow included verification steps where GPT-5.2 scanned for subclasses of `ObjectPermission` and validated that `WeblogPermission` does not directly access encapsulated fields, confirming backward compatibility.

The model also updated `WeblogPermission` to:

- Use the newly encapsulated accessor methods `setObjectId`, `setObjectType`, and `setUserName` instead of direct field assignment.
- Ensure compatibility with the refactored base class.

After implementing changes, the agentic workflow executed Maven compile commands with specific module and test selections to verify successful compilation without breaking existing builds. The final output explicitly confirmed build success, demonstrating that the refactoring was both design-sound and practically deployable.

The key advantage of the agentic approach was its ability to handle cross-file dependencies and validate assumptions through testing, something impossible in single-shot non-agentic mode. GPT-5.2 in agentic mode successfully navigated the complexity of a multi-class refactoring, ensuring that changes to the base class did not introduce regressions in derived classes. This represents a substantial improvement in both correctness and safety compared to non-agentic refactoring.

Claude Sonnet 4.5 Non-Agentic Mode

Design Smell Identification

Claude Sonnet 4.5 in non-agentic mode identified six design smells, demonstrating comprehensive file-level analysis with precise categorization.

- The model detected Incomplete Abstraction, noting that `ObjectPermission` is declared abstract but has commented-out methods for `getObjectType` and `setObjectType`, and the `objectType` field is declared but never properly exposed or used, creating incomplete abstraction where the class's interface does not fully represent its internal state.
- Claude also identified Primitive Obsession for Deficient Encapsulation, recognizing that the `actions` field is represented as a raw `String` without validation, parsing, or type safety, and `actions` should be encapsulated in a more meaningful type.
- Additionally, Claude flagged Broken Modularization, noting that the class has field initialization logic scattered between field declarations with inline initialization of `id`, `pending`, and `dateCreated`, and constructors, creating inconsistent state initialization patterns.
- The model detected Unnecessary Abstraction, observing that the class is abstract but provides no abstract methods, making the abstraction questionable since if there is no required contract for subclasses to implement, the abstraction may be unnecessary.
- Claude also identified Deficient Encapsulation in two locations:
 - The `dateCreated` field and `setDateCreated` method, where the `Date` object is mutable and returning the internal reference lets outside code mutate internal state without calling setters.
 - The `setDateCreated` method, which similarly accepts a reference and allows external code to modify the internal date, violating encapsulation principles.
- Finally, Claude detected Broken Hierarchy in the inheritance relationship with `RollerPermission`, noting that the constructors call `super` with empty strings or the `name` parameter, but `ObjectPermission` class does not meaningfully use the `name` concept from the parent, suggesting a potentially broken or awkward inheritance relationship.

This analysis is particularly notable because Claude identified the Broken Hierarchy smell even in non-agentic mode, demonstrating superior architectural reasoning compared to GPT-5.2. However, like GPT-5.2, Claude in non-agentic mode did not detect subsystem-level issues such as Cyclic-Dependent Modularization, which require multi-file context.

Refactoring Quality and Compilability

Claude Sonnet 4.5 produced high-quality refactored code that addressed all six identified smells while maintaining compilation safety.

- The refactoring removed commented-out code and properly implemented the `objectType` methods, completing the abstraction.

- Claude created an `Actions` enum class to encapsulate valid action types, centralizing parsing, serialization, and validation, and replacing the primitive `String` representation with a more meaningful type.
- The model also moved all initialization logic to constructors, implementing a consistent initialization pattern, and added a parameterized constructor for cleaner object creation.
- To address the Unnecessary Abstraction smell, Claude introduced an abstract method `hasAction` that subclasses must implement to provide a clear contract for subclasses, establishing proper polymorphic behavior.
- For Deficient Encapsulation, the model implemented defensive copying, where the getter returns a copy of the `Date` and the setter accepts a reference but stores a copy, preventing external mutation from changing internal state.
- Claude also addressed the Broken Hierarchy issue by properly using the parent's name concept, ensuring constructors establish valid object state and align with the inheritance model.

The refactored code is syntactically valid, preserves the original API for compatibility, and introduces new abstractions such as the `Actions` class that enhance design quality without breaking existing functionality. However, similar to GPT-5.2, Claude's refactoring includes the risk that changing protected fields to private could break subclasses that directly access these fields.

The key difference is that Claude explicitly recognized and attempted to address the Broken Hierarchy issue, suggesting improved constructor semantics, whereas GPT-5.2 in non-agentic mode did not detect this smell at all. Overall, Claude's non-agentic refactoring demonstrates superior design awareness and more principled improvements.

Claude Sonnet 4.5 Agentic Mode

Design Smell Identification

Claude Sonnet 4.5 in agentic mode exhibited the most comprehensive and architecturally aware analysis of all four tested configurations. The model identified **nine distinct design smells** through multi-step reasoning and cross-file inspection.

Beyond the smells detected in non-agentic mode, the agentic workflow enabled Claude to identify three additional high-severity architectural issues:

- **Broken Encapsulation via Protected Fields**
 - All seven instance fields declared as protected allow direct access and modification from any subclass.
 - Subclasses like `WeblogPermission` directly assign to these fields, creating tight coupling.
- **Mutable Date Object Exposure**
 - The getter returns the internal `Date` reference directly without defensive copying.

- The setter accepts a reference and stores it, allowing external callers to mutate internal state.
- **Incomplete API Design**
 - The `objectType` field is protected and accessed by subclasses.
 - Its getter and setter methods are commented out, creating API inconsistency where subclasses can access the field directly but external code cannot query it through public methods.

Claude also detected additional architectural weaknesses:

- **Field Initialization with Side Effects**
 - The `id` field is initialized with `UUIDGenerator.generateUUID` at the point of declaration.
 - This causes issues with JPA and persistence frameworks and any code relying on default constructors producing objects without side effects.
- **Missing equals and hashCode in Base Class**
 - The base class contains an `id` field that appears to be a unique identifier but does not override `equals` and `hashCode`.
 - While the subclass `WeblogPermission` implements these methods, the base class does not, leading to inconsistent behavior in polymorphic usage or collections.
- **Inconsistent Object Initialization**
 - Some fields are initialized inline while others are left null.
 - Constructors do not establish valid object state, leaving initialization responsibility to subclasses or setters.
- **Weak Hierarchy Design**
 - The constructor with name parameter passes it to the superclass `RollerPermission`, but its purpose is unclear and not stored in the class.
 - Subclasses use it to construct descriptive names, but the base class does not make this contract explicit.

This collection of smells represents the most thorough and architecturally comprehensive analysis of any tested configuration, demonstrating Claude's superior ability to reason about design quality when equipped with agentic capabilities.

Refactoring Quality and Compilability

Claude Sonnet 4.5 in agentic mode produced the most comprehensive and architecturally sound refactoring of all tested configurations. The refactoring addressed all nine identified design smells through coordinated changes across `ObjectPermission` and `WeblogPermission`.

Encapsulation Improvements

- Changed all protected fields to private.
- Added proper getter and setter methods with appropriate access levels.

- Protected setters for `id`, `objectType`, and `objectId` allow controlled subclass initialization.
- Enhanced documentation clarifies subclass extension points.

Mutable Date Fix

- Implemented defensive copying in `getDateCreated` (returns copy).
- Implemented defensive copying in `setDateCreated` (stores copy).

API Completion

- Uncommented and properly exposed `getObjectType` and `setObjectType` if actively used.
- Alternatively removed `objectType` entirely if functionally unnecessary.

Removal of Field Initialization Side Effects

- Moved `UUIDGenerator.generateUUID` from field declaration to constructor body.
- Added a no-argument constructor for JPA and persistence frameworks without UUID side effects.
- Allowed frameworks to construct objects and populate fields safely.

Equality and Identity Consistency

- Implemented `equals` and `hashCode` based on `id` as natural identifier.
- Used `Objects.equals` and `Objects.hash` for null-safe operations.
- Ensured consistent behavior across the inheritance hierarchy.

Initialization Consistency

- Consolidated initialization logic into `initializeDefaults` method.
- Added parameterized constructors to ensure complete initialization.
- Introduced factory methods (e.g., `createNewEntityPermission`) where appropriate.

Hierarchy Simplification

- Removed confusing constructor with `name` parameter.
- Implemented a simplified default constructor with explicit initialization.
- Ensured the `name` passed to superclass is meaningful and consistently used.

Cross-Class Compatibility

- Updated `WeblogPermission` to use encapsulated access via `setObjectId`, `setObjectType`, and `setUserName` instead of direct field access.
- Simplified subclass constructors to align with the refactored base class.

Verification and Build Safety

- Verified successful compilation through iterative Maven builds.
- Ensured backward compatibility with existing behavior.
- Confirmed that field names remain unchanged for IPA queries and persistence compatibility.
- Validated that the code compiles successfully without errors.

The final refactored code demonstrates complete design smell remediation while maintaining full compatibility with the existing Apache Roller platform. This represents the highest quality refactoring output of all tested configurations, combining comprehensive smell detection with safe, validated, and architecturally sound remediation.

Comparative Analysis

Design Smell Detection Capability

The ability to identify design smells varied significantly across models and modes.

- GPT-5.2 in non-agentic mode detected four smells, all accurately identified and correctly categorized, but limited to file-level patterns.
- Claude Sonnet 4.5 in non-agentic mode detected six smells, including the critical Broken Hierarchy smell that GPT-5.2 missed, demonstrating superior architectural reasoning even without multi-file context.
- In agentic mode, GPT-5.2 detected seven smells through cross-file analysis and iterative inspection.
- Claude Sonnet 4.5 detected nine smells with the most comprehensive architectural coverage.

Compared to our manual analysis baseline, which identified Deficient Encapsulation, Incomplete Abstraction, and Broken Hierarchy at the file level:

- Both models in non-agentic mode successfully detected Deficient Encapsulation and Incomplete Abstraction.
- Only Claude detected Broken Hierarchy in non-agentic mode, while GPT-5.2 required agentic capabilities to identify hierarchy issues.
- Neither model in any mode detected the subsystem-level Cyclic-Dependent Modularization, which requires understanding interactions across the entire permission subsystem spanning multiple packages.

The key finding is that agentic workflows substantially improve architectural awareness for both models, enabling detection of cross-class issues invisible in single-file analysis. Claude consistently demonstrated superior design smell detection in both modes, identifying more smells and showing better architectural reasoning. The gap between Claude and GPT-5.2 narrowed in agentic mode, but Claude maintained an advantage in comprehensiveness and precision.

Refactoring Quality and Compilability

All four configurations produced syntactically valid and compilable code, but they differed significantly in completeness, safety, and architectural soundness.

- GPT-5.2 in non-agentic mode produced clean refactorings that addressed all detected smells but included the risk of breaking subclasses through protected-to-private field changes without verification.
- Claude in non-agentic mode produced higher-quality refactorings with more principled improvements, including the introduction of type-safe abstractions such as the Actions enum and explicit handling of the Broken Hierarchy issue through improved constructor semantics.

In agentic mode, both models demonstrated substantial improvements:

- GPT-5.2 produced comprehensive refactorings with iterative testing and cross-file validation, ensuring that changes to ObjectPermission did not break WeblogPermission, and explicitly verified build success through Maven compilation.
- Claude in agentic mode produced the highest-quality refactorings with the most thorough smell remediation, complete cross-class coordination including defensive copying, proper initialization patterns, equals and hashCode implementation, and API completeness, and rigorous validation ensuring backward compatibility with existing IPA queries and successful compilation without errors.

The critical advantage of agentic workflows was their ability to validate assumptions through testing and handle cross-file dependencies safely. Non-agentic refactorings, while syntactically correct, carried inherent risks because the models could not verify whether subclasses would break. Agentic workflows eliminated this risk through iterative validation and cross-file updates. Claude's agentic refactorings were superior to GPT-5.2's, demonstrating more comprehensive design improvements and more rigorous validation.